

WEEK-8

TASK-1:

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

CODE:

```
#include <stdio.h>

#include <stdlib.h>

int main()
{
    int i;

    printf("==== DYNAMIC MEMORY ALLOCATION =====\n");

    // 1. malloc()

    int *arr1 = (int *)malloc(3 * sizeof(int));

    if (arr1 == NULL)
    {
        printf("malloc failed\n");

        return 1;
    }

    printf("\n1. malloc() allocated memory for 3 integers\n");

    for (i = 0; i < 3; i++)
    {
        arr1[i] = (i + 1) * 10;
    }

    printf("malloc values: ");

    for (i = 0; i < 3; i++)
    {
        printf("%d ", arr1[i]);
    }
}
```

```
printf("\n");  
  
// 2. calloc()  
int *arr2 = (int *)calloc(3, sizeof(int));  
if (arr2 == NULL)  
{  
    printf("calloc failed\n");  
    free(arr1);  
    return 1;  
}  
printf("\n2. calloc() allocated memory for 3 integers\n");  
printf("calloc initial values: ");  
for (i = 0; i < 3; i++)  
{  
    printf("%d ", arr2[i]);  
}  
printf("\n");  
  
// 3. realloc()  
arr1 = (int *)realloc(arr1, 5 * sizeof(int));  
if (arr1 == NULL)  
{  
    printf("realloc failed\n");  
    free(arr2);  
    return 1;  
}  
printf("\n3. realloc() resized memory from 3 to 5 integers\n");  
arr1[3] = 40;  
arr1[4] = 50;  
printf("After realloc: ");  
for (i = 0; i < 5; i++)
```

```

{
    printf("%d ", arr1[i]);
}

printf("\n");

free(arr1);

free(arr2);

printf("\n4. free() released all allocated memory\n");

printf("\nMemory allocation program completed successfully.\n");

return 0;
}

```

```

==2174== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
root@LAPTOP-RJFD8EBT:~# nano dynamic_memory.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g dynamic_memory.c -o dynamic_memory
root@LAPTOP-RJFD8EBT:~# valgrind --leak-check=full --show-leak-kinds=all ./dynamic_memory
==2174== Memcheck, a memory error detector
==2174== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2174== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2174== Command: ./dynamic_memory
==2174==
===== DYNAMIC MEMORY ALLOCATION =====

1. malloc() allocated memory for 3 integers
malloc values: 10 20 30

2. calloc() allocated memory for 3 integers
calloc initial values: 0 0 0

3. realloc() resized malloc memory from 3 to 5 integers
After realloc: 10 20 30 40 50

4. free() released all allocated memory

Memory allocation program completed successfully.
==2174==
==2174== HEAP SUMMARY:
==2174==   in use at exit: 0 bytes in 0 blocks
==2174==   total heap usage: 4 allocs, 4 frees, 1,068 bytes allocated
==2174==
==2174== All heap blocks were freed -- no leaks are possible
==2174==
==2174== For lists of detected and suppressed errors, rerun with: -s
==2174== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@LAPTOP-RJFD8EBT:~#

```

TASK-2:

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

CODE:

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <string.h>

```

```

#define SIZE (10 * 1024 * 1024) // 10 MB

char data[SIZE];

void show_memory(const char *message)
{
    char command[200];
    printf("\n%s\n", message);
    snprintf(command, sizeof(command),
              "grep -E 'Rss:|Pss:|Shared_Dirty:|Private_Dirty:'
/proc/%d/smmaps_rollup",
              getpid());
    system(command);
}

int main()
{
    pid_t pid;
    printf("==== COPY-ON-WRITE DEMONSTRATION =====\n");
    memset(data, 'A', SIZE);
    printf("\nParent PID: %d\n", getpid());
    show_memory("Parent memory BEFORE fork():");
    pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    if (pid == 0)

```

```

{
    printf("\nChild PID: %d\n", getpid());

    show_memory("Child memory BEFORE modification:");

    printf("\nChild is modifying 5 MB of shared data...\n");

    memset(data, 'B', SIZE / 2);

    show_memory("Child memory AFTER modification:");

    printf("\nChild data[0] = %c\n", data[0]);

    exit(0);
}

else
{
    wait(NULL);

    printf("\nParent data[0] = %c\n", data[0]);

    show_memory("Parent memory AFTER child modification:");

    printf("\nParent remains unchanged because of Copy-on-Write.\n");
}

return 0;}

```

```

root@LAPTOP-RJFD8EBT:~# nano cow_demo.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g cow_demo.c -o cow_demo
root@LAPTOP-RJFD8EBT:~# ./cow_demo
===== COPY-ON-WRITE DEMONSTRATION =====
Parent PID: 2208
Parent memory BEFORE fork():
Rss: 11980 kB
Pss: 10818 kB
Shared_Dirty: 0 kB
Private_Dirty: 10352 kB
SwapPss: 0 kB
Child PID: 2211
Child memory BEFORE modification:
Rss: 11308 kB
Pss: 5401 kB
Shared_Dirty: 10308 kB
Private_Dirty: 40 kB
SwapPss: 0 kB
Child is modifying 5 MB of shared data...
Child memory AFTER modification:
Rss: 11308 kB
Pss: 7963 kB
Shared_Dirty: 5184 kB
Private_Dirty: 5164 kB
SwapPss: 0 kB
Child data[0] = B
Parent data[0] = A
Parent memory AFTER child modification:
Rss: 11980 kB
Pss: 10818 kB
Shared_Dirty: 0 kB
Private_Dirty: 10352 kB
SwapPss: 0 kB
Parent remains unchanged because of Copy-on-Write.
root@LAPTOP-RJFD8EBT:~# fork();
-bash: syntax error near unexpected token ';'
root@LAPTOP-RJFD8EBT:~# cat /proc/2208/status | grep -E 'VmSize|VmRSS|VmData'
cat: /proc/2208/status: No such file or directory
root@LAPTOP-RJFD8EBT:~#

```